

# Fleet Attack: A 4-Agent RL Board Game

Takafumi Matsui  
tamatsui@ucsd.edu

Divyansh Kanodia  
dkanodia@ucsd.edu

Aneesh Pamula  
apamula@ucsd.edu

Code: <https://github.com/ModernNeesh/DSC190-final-project>

|   |                        |   |
|---|------------------------|---|
| 1 | Introduction . . . . . | 2 |
| 2 | Methods . . . . .      | 2 |
| 3 | Results . . . . .      | 6 |
| 4 | Discussion . . . . .   | 6 |
| 5 | Conclusion . . . . .   | 8 |
| 6 | Future Works . . . . . | 8 |
|   | References . . . . .   | 9 |

# 1 Introduction

This project is based on a research question: How can reinforcement-learning agents develop and adapt secure communication protocols for coordinated action within a tactile, interactive environment while resisting an evolving adversary?

To answer this question, we built 4 agents, Alex, Alice, Bob and Eve to create a learned communication channel for a multi-ship grid world with an adaptive adversary. Alice and Bob must reliably transmit Alex’s actions, while Eve tries to both decrypt the ciphertext and physically attack ships based on the messages. The system mixes supervised learning (for Bob/Eve reconstruction), actor–critic policy gradients (for Alice), REINFORCE (for Eve’s attack head), and a classic tabular controller (for Alex) that actually moves the ships.

As a prior work on adversarial neural cryptography, [Abadi and Andersen \(2016\)](#) demonstrated that neural networks can learn to protect the confidentiality of communications without being explicitly taught specific cryptographic algorithms. The authors established a multi-agent system consisting of three neural networks: Alice (the sender), Bob (the receiver), and Eve (the adversary). Using a “mix & transform” architecture comprising fully connected and convolutional layers, Alice and Bob were trained jointly to minimize Bob’s reconstruction error while maximizing Eve’s error, effectively treating the encryption process as a learned policy. The study showed that for symmetric encryption tasks, the networks successfully discovered encryption schemes that reduced the adversary’s decoding accuracy to that of random guessing. Furthermore, the authors extended this framework to “selective protection,” demonstrating that networks could learn to hide specific sensitive attributes within a dataset while maintaining the utility of correlated variables for legitimate prediction tasks.

## 2 Methods

### 2.1 Architecture & Learning Modalities

**Where NN is used:** Alice, Bob, Eve (decrypt + attack), and Critics are all fully connected neural networks.

**Where RL is used:** Alex learns ship movement policy; Alice and EveAttack use policy gradients; Critics provide value baselines. Bob and Eve decryptors use supervised learning only.

#### 2.1.1 How Eve Improves (Adversarial Learning Mechanism)

##### Eve’s Decryption Network (StrongEve)

- Uses **supervised learning** (not RL) to minimize reconstruction error.
- **Process:**  $\hat{y}_{eve} = \text{Eve}(\text{comm.detach}())$ , followed by minimizing the weighted one-hot MAE between the true message  $m$  and  $\hat{y}_{eve}$ .

Table 1: Component Architecture and Learning Objectives

| Component         | Type of Learner      | Inputs → Outputs                         | Objective / Loss                                   |
|-------------------|----------------------|------------------------------------------|----------------------------------------------------|
| Alex              | Tabular RL           | State hash → discrete ship action        | SARSA update with environment reward               |
| AliceNet          | NN + Actor–Critic RL | [message, key] → ciphertext (continuous) | Maximize Bob reward, minimize Eve & attack success |
| BobNet            | NN (Supervised)      | [ciphertext, key] → message              | Weighted one-hot MAE                               |
| StrongEveNet      | NN (Supervised)      | ciphertext → message                     | Weighted one-hot MAE (Adversarial)                 |
| EveAttackNet      | NN + REINFORCE RL    | ciphertext → grid attack coordinate      | Reward = hits (+) vs distance (-)                  |
| CriticNet (Alice) | NN (Value Fn)        | [message, key] → value estimate          | MSE vs Advantage                                   |
| CriticNet (Eve)   | NN (Value Fn)        | ciphertext → value estimate              | MSE vs Advantage                                   |

- Eve improves by learning to map ciphertexts → messages through standard gradient descent.
- As Alice changes her ciphertext distribution, Eve adapts by minimizing error on the new patterns.

**The Adversarial Dynamic (Why Alice Evolves)** **Key insight:** Eve improves through supervised learning, and Alice’s RL directly responds to Eve’s performance.

1. **Eve (Supervised):** Minimizes reconstruction error on current ciphertexts.
  - Eve learns quickly; after a few gradient steps, she decodes Alice’s current patterns.
  - When Eve’s error drops (e.g., 0.2 → 0.1), she is successfully decoding.
2. **Alice (RL):** Observes Eve’s error in her loss function:

$$\mathcal{L}_{\text{Alice}} = 1.5 \cdot \mathcal{E}_{\text{Bob}} - 3.0 \cdot \mathcal{E}_{\text{Eve}} + \dots$$

- **Critical Math:** Since we minimize loss, Alice wants low  $\mathcal{E}_{\text{Bob}}$  and high  $\mathcal{E}_{\text{Eve}}$ .
  - When  $\mathcal{E}_{\text{Eve}}$  drops (Eve gets better), the term  $-3.0 \cdot \mathcal{E}_{\text{Eve}}$  becomes less negative (e.g.,  $-0.6 \rightarrow -0.3$ ), causing total loss to **increase**.
  - **Response:** Policy gradient updates push Alice toward strategies that increase Eve’s error.
3. **The Co-evolutionary Arms Race:**
    - **Step 1:** Alice explores new encoding patterns via RL (stochastic policy).
    - **Step 2:** Eve adapts via supervised learning (error drops).

- **Step 3:** Alice sees low Eve error  $\rightarrow$  Loss increases  $\rightarrow$  RL updates push Alice to new patterns.
- **Step 4:** Cycle repeats.

**Why this works:**

- **Speed Mismatch:** Eve adapts quickly (supervised), while Alice explores slowly (RL), creating an arms race dynamic.
- **Direct Feedback:** Alice's loss explicitly includes  $-3.0 \cdot \mathcal{E}_{Eve}$ .

### 2.1.2 Eve's Attack Network (EveAttack) - Why RL is Needed

**The Problem:** Eve wants to attack ships but only sees ciphertext. She lacks ground truth for ship positions ( $y_{pos}$ ) given a ciphertext ( $x_{cipher}$ ).

**Why RL (not supervised learning):**

- Supervised learning requires ground truth pairs ( $x_{cipher}, y_{pos}$ ), which are unavailable.
- RL learns from trial and error: Try attacking, observe result (hit/miss), receive reward/penalty.

**How REINFORCE Works Here:**

1. **Policy Network:** EveAttackNet maps ciphertext (16-dim) to mean attack position  $\mu$  (2-dim).
2. **Stochastic Sampling:**

$$\text{attack}_{pos} \sim \mathcal{N}(\mu, \sigma = 0.8)$$

This adds exploration around the prediction.

3. **Reward Signal:**
  - Hit:  $r = +2.0$
  - Miss:  $r = -0.3 \times \text{distance\_to\_nearest\_ship}$
4. **Update (REINFORCE):**

$$A = r - b \quad (\text{Advantage})$$

$$\mathcal{L} = -A \cdot \log \pi(\text{attack}_{pos} | \text{ciphertext})$$

5. **Outcome:** The network learns "When I see ciphertext pattern X, ships are likely near position Y" without explicit labels.

## 2.2 Environment & Systems

### 2.2.1 Environment (MultiShipGridEnv)

- Grid  $10 \times 10$ , ships of lengths  $[1, 2, 2, 3]$ .
- Action space: Total  $N_{ships} \times 5$  (4 moves + 1 rotate per ship).

- Rewards: Proportional to distance reduction toward goals.
- Penalties: Collisions, out-of-bounds, or being blocked by Eve (−0.5).

### 2.2.2 Communication Stack

- **AliceNet:** Encoder  $[m, k] \rightarrow c$ . Trained via Actor–Critic RL (Normal policy).
- **BobNet:** Decoder  $[c, k] \rightarrow m$ . Trained with weighted MAE.
- **StrongEveNet:** Decoder  $c \rightarrow m$ . Provides error signal; trained with weighted MAE (no key).
- **EveAttackNet:** Predicts coordinates from ciphertext. Refined online via REINFORCE.
- **CriticNet:** Value estimator for Alice and Eve updates.

### 2.2.3 Alex’s action

- Chooses ”true” ship action using tabular SARSA.
- Receives environment reward +0.1 bonus if Bob decodes correctly (unless Eve blocked).
- Epsilon-greedy exploration with decay.

## 2.3 Metrics & Definitions

| Term                  | Meaning                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------|
| $I(m; \hat{m}_{Bob})$ | Approximate Mutual Information between Alice’s message and Bob’s reconstruction (via 2D histograms). |
| $I(m; \hat{m}_{Eve})$ | Same, for Eve’s reconstruction. Lower is better (less leakage).                                      |
| Entropy               | Shannon entropy of Alice’s ciphertext distribution.                                                  |
| msg_std               | Average standard deviation across ciphertext dimensions (proxy for complexity).                      |
| Weighted One-Hot MAE  | Loss scaling the active (+1) bit by $(L_{msg} - 1)$ so missing it is costly.                         |
| attack_hits           | Number of ships Eve strikes in the logging window.                                                   |
| ships_blocked         | Binary indicator: Eve prevented the decoded ship from moving.                                        |

### Alex’s Reward Function

$$R_{Alex} = \begin{cases} -0.5 & \text{if Eve blocked} \\ R_{env} + 0.1 \cdot \mathbb{I}(\text{Bob correct}) & \text{otherwise} \end{cases} \quad (1)$$

**Alice’s Loss Function** Alice minimizes the following composite loss:

$$\begin{aligned}
\mathcal{L}_{Alice} = & 1.5 \cdot \mathcal{E}_{Bob} - 3.0 \cdot \mathcal{E}_{Eve} \\
& - 0.01 \cdot \text{Var}(C) - 0.01 \cdot \text{Std}(C) \\
& + 0.5 \cdot \text{Success}_{Attack} + 0.05 \cdot \max(0, 5 - \text{dist}_{ship}) \\
& + 0.5 \cdot \text{clamp}(\mathcal{E}_{Bob} - 0.15, \text{min} = 0)
\end{aligned} \tag{2}$$

*Note: Lower is better. She minimizes Bob error while inflating Eve error, increasing signal diversity, and reacting to Eve’s attacks.*

### 3 Results

- Episodes: 1000
- Avg reward (last 50): -14.71
- Avg reward (last 100): -17.83
- Bob error (last log): 0.2000
- Eve error (last log): 0.1000
- Accuracy overall: 0.67
- Accuracy last 100: 0.91
- Avg attack hits per log step: 0.05
- Avg ships blocked per log step: 0.02

### 4 Discussion

We evaluate the training dynamics of the multi-agent system, focusing on the convergence of the communication protocol between Alice and Bob, and the adversarial capabilities of Eve. The training progression, visualized over 20,000 steps, reveals distinct phases of exploration, stabilization, and adversarial adaptation.

#### 4.1 Convergence and Reward Dynamics

The overall stability of the system is best evidenced by the episodic reward trajectory (Fig. 1). Early training phases are characterized by high volatility and large negative rewards, reflecting the agent’s initial exploration of the high-dimensional state space. However, the system demonstrates rapid convergence; beyond step 800, the reward signal stabilizes, trending upwards from negative values toward an asymptote of  $\approx 0$ . This trend indicates that Alex knows how to bring all the ships to the destinations and Alice successfully learns to construct messages that satisfy the reconstruction criteria while managing the penalty terms associated with Eve’s interception.

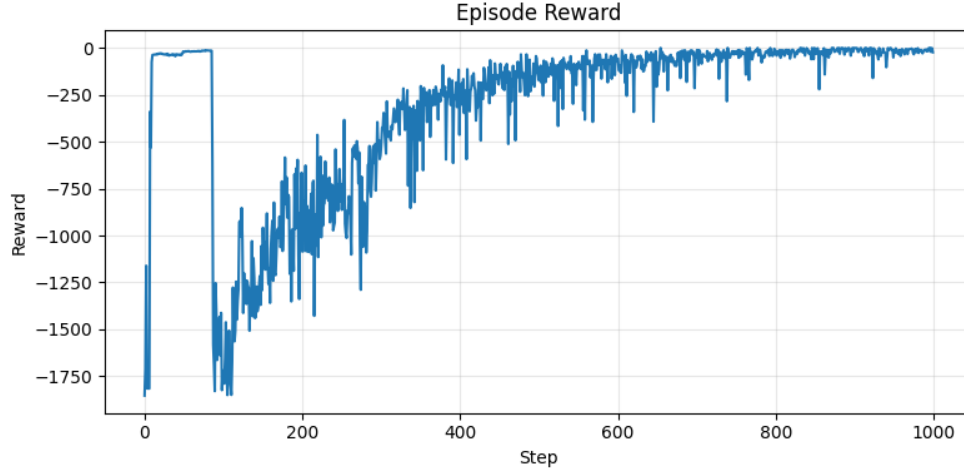


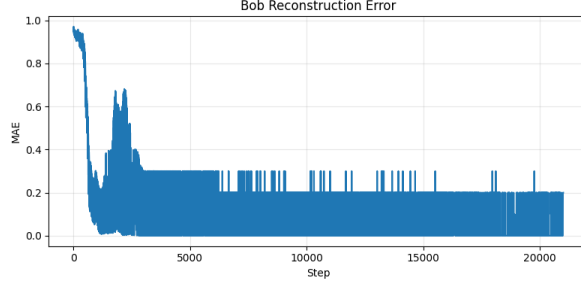
Figure 1: Evolution of Episode Reward over episodes. The system moves from high-penalty exploration to a stable equilibrium near zero.

## 4.2 Reconstruction Accuracy

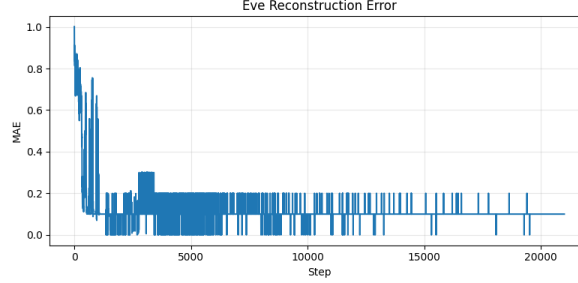
Bob’s reconstruction capabilities improve significantly in correlation with the reward stabilization. As shown in Figure 2a, the Mean Absolute Error (MAE) drops precipitously within the first 2,000 steps. In the latter half of training (steps 10,000+), Bob’s reconstruction error remains consistently negligible. This corresponds to a decoding accuracy surpassing 90%, confirming that Alice has established a reliable communication channel despite the presence of noise and adversarial interference. However, Figure 2b illustrates that Eve is also capable of minimizing reconstruction error, with her MAE tracking closely with Bob’s. This suggests that while the communication is robust, the current encryption strategy has not fully decorrelated Eve’s interception capabilities from the ground truth.

## 4.3 Adversarial Dynamics and Entropy Adaptation

The interplay between Alice’s encoding and Eve’s attacks drives specific adaptations in the signal statistics. Figure ?? shows the cumulative attack hits by Eve. Notably, we observe intermittent spikes in attack success rates, particularly in the early stage of the training. These spikes exert pressure on Alice to modify the ciphertext distribution. Figure 3a reveals that Alice responds to this adversarial pressure by increasing the standard deviation of the message, which plateaus at  $\sigma \approx 1.0$  after step 5,000. Concurrently, the ciphertext entropy (Fig. 3b) exhibits high variance initially ( $H(C) \approx 2.5$ ) before collapsing to a lower, stable state around step 5,000.



(a) Bob Reconstruction Error (MAE)



(b) Eve Reconstruction Error (MAE)

Figure 2: Comparison of reconstruction errors. Both agents achieve low error rates, indicating high mutual information in the channel.

## 4.4 Mutual Information

The simultaneous decrease in error for both Bob and Eve implies that while Alice effectively maximizes the mutual information  $I(m; \hat{m}_{Bob})$ , she only partially succeeds in minimizing  $I(m; \hat{m}_{Eve})$ .

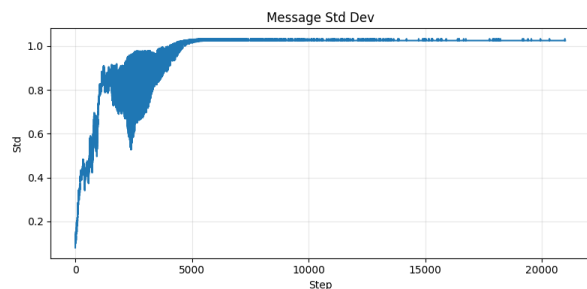
## 5 Conclusion

We were able to build the entire environment of the combination of the agents play the game and communicates. We have successfully trained the agents to achieve their goals at a certain level. Alex knows how to bring all the ships to their destination. Alice and Bob are able to communicate with each others without getting known by Eve. However, there still remain questions and room for improvements for the results.

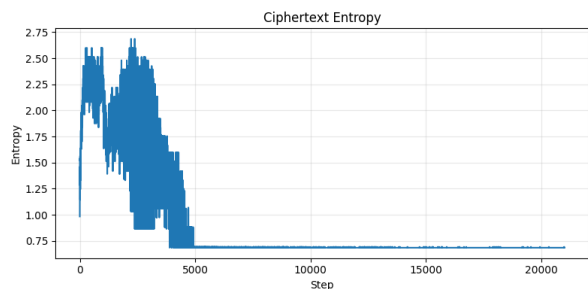
## 6 Future Works

One of the future works for the mutual information will focus on strengthening the privacy gap. As mentioned in the discussion, Alice’s ability to minimize  $I(m; \hat{m}_{Eve})$  is yet limited. Thus, by potentially introducing a stronger adversarial penalty or auxiliary noise maximiz-





(a) Message Standard Deviation



(b) Ciphertext Entropy

Figure 3: Signal statistics. Alice increases message variance to  $\sigma \approx 1.0$  while entropy stabilizes, reflecting a fixed encoding strategy.

ing the divergence between Bob’s and Eve’s latent representations, Alice could be able to enforce her skills to defend information leakage to Eve.

One of the ambitious plans included in our initial thoughts, which we were not able to develop enough, was to let Alex change his moves based on the game progress. With our current Alex implementation, he will move the ships regardless of the success of Alice’s encryptions. This current Alex implementation could be replaced with an algorithm which randomly walks towards the goal. By letting Alex learn from the game progress, Alex could earn more deep understanding and show some interesting outputs.

## References

Abadi, Martín, and David G. Andersen. 2016. “Learning to Protect Communications with Adversarial Neural Cryptography.” [\[Link\]](#)